

AI2611: Machine Learning Course Assignment

Mac-SVMs: Modern Accelerated SVM Series

Kailing Wang (Email)
Shanghai Jiao Tong University

2023.5.15

Abstract—In this report, I will explain the details of my SVM implementation, the novel acceleration methods I used during implementation and my results on dataset MNIST and CIFAR10. The proposed version of my single-kernel model ...multi-kernel model

Index Terms—SVM, CUDA, SGD, Classification, Multi-Kernel, PCA

I. INTRODUCTION

When SVM was first proposed [1], SMO(Sequential Minimal Optimization) was used to optimize the SVM objective function. However, SMO requires us to feed the data pair by pair and will cause $O(n^2)$ time complexity for a single iteration. Moreover, the SMO algorithm does not support parallel computing. Modern-day computing devices are good at processing multiple tasks and performing matrix calculations. Considering that the SVM method is no more than solving an optimization problem, there are later more and more methods [2]–[5] using gradient descend inside SVM and the batch-based version SGD should make the most of modern computing device. However, some doubt whether the SGD method can be used in SVM. Domingos et al. claim that every model learned by gradient descent is approximately a kernel machine [6], and provided detailed mathematical derivation. In fact, the SVM in sklearn is also implemented with SGD model.

II. TASK ANALYSIS

The task is to implement SVM from scratch and use SVM models to perform classifications on MNIST and CIFAR10. First, consider the features of the datasets: I expect a relatively good result on MNIST because it is obviously almost linear and separable. MNIST images can be viewed as true or false maps. Fig 1 shows the pixel frequency of all the 0s(left) and 1s(right) in MNIST. As we can see, there exist some pixels that are likely to be reached when writing a number but not as likely writing another. Using SVM, we should have good results even using raw images flattened into vectors. Moreover, since some pixels are reached by almost every number and some are never reached, we should desert these pixels. In fact, this can make the data vector significantly smaller, and I've tested using only 10 selected pixels is enough for a 0-1 classification. However, for CIFAR10 the problem is not that



Fig. 1. All 0s and 1s stacked in MNIST

simple. Images in CIFAR10 have no obvious feature if viewed raw. A car can be all colors, and thus may be similar to a cat in Euclidean space. We must consider extracting further features to expect better performance.

III. METHODS

A traditional SVM objective function is

$$\begin{aligned} \min_{w, b} \quad & \phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_i \xi_i \\ \text{st.} \quad & y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \xi_i \\ & \xi_i \geq 0 \end{aligned} \quad (1)$$

Usually, we use the KKT condition to solve the problem,

$$\begin{aligned} J(\mathbf{w}, b, \alpha, \beta, \xi) &= \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \sum_i \xi_i - \sum_i \beta_i \xi_i \\ &\quad - \sum_i \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i) \\ \frac{\partial J}{\partial \mathbf{w}} &= 0, \quad \frac{\partial J}{\partial b} = 0, \quad \frac{\partial J}{\partial \xi_i} = 0 \\ y_i(\mathbf{w}^T \mathbf{x}_i + b) &\geq 1 - \xi_i \\ \xi_i &\geq 0, \quad \alpha_i \geq 0, \quad \beta_i \geq 0 \\ \alpha_i (y_i(\mathbf{w}^T \mathbf{x}_i + b) - 1 + \xi_i) &= 0 \\ \beta_i \xi_i &= 0 \end{aligned} \quad (2)$$

I'm not going to type the dual problem or other equations concerning SMO here. Well, I'm just to show how complex is

the optimization process, and in case you doubt if I'm aware of the SMO method. SMO is believed to converge on a good result though. But as I've mentioned, the fatal problem of SMO is that no matter what searching methods are used to select multipliers, we step pair by pair. This is not elegant at all in modern days. The time cost can explain everything: the code copied from the Internet by some takes half an hour to calculate on CIFAR10.

A. Simplified Object Function

Before I start doing this assignment, some told me it takes a long time even to run on MNIST. From that on I decided to take an effort to simplify and accelerate.

From equation 1, we should recognize that

$$\xi_i \geq \max(1 - y_i(\mathbf{w}^T \mathbf{x}_i + b), 0)$$

, Thus, optimizing equation 1, we are about to optimize

$$\min_{w,b} \phi(\mathbf{w}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + C \max(1 - y_i(\mathbf{w}^T \mathbf{x}_i + b), 0) \quad (3)$$

This is learned in the Convex Optimization class. If we minimize the max part, we are also minimizing the original problem since we can choose the lower bound as ξ .

Kernel Functions

I use some of the most basic kernel functions. Designing a kernel function is additional work to our task.

1) *Linear*: Do nothing, if we view kernel functions as a transform of multiplication.

$$K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j \quad (4)$$

2) *Poly*:

$$K(\mathbf{x}_i, \mathbf{x}_j, c, d) = (\mathbf{x}_i^T \mathbf{x}_j + c)^d \quad (5)$$

3) *RBF(or Gaussian)*:

$$K(\mathbf{x}_i, \mathbf{x}_j, \gamma) = e^{-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|_2} \quad (6)$$

We can also rewrite this equation like a Gaussian function and use parameter σ . But it's still the same function.

B. SGD Method

Now, we have an unrestricted problem, and we're ready to perform SGD.

Algorithm 1: SGD SVM

```

1 Initialize the  $w$  and  $b$ .
2 Define a kernel function  $K(x_i, x_j)$ .
3 for each iteration  $t$  do
4   Select a batch  $B = (x_i, y_i)$  from the training set.
5   Compute margin:  $\text{margin} = y_i \cdot (K(B_X, w) + b)$ .
6   Calculate the gradients of  $w$  and  $b$ .
7   Update  $w$  and  $b$  using the learning rate  $\eta$ :
8 Use  $w$  and  $b$  to predict new data:
   prediction =  $\text{sign}(K(x_{\text{new}}, w) + b)$ 

```

However, since there is a max function in the objective function, the gradient is not continuous in the border where the margin is exactly 1. And when the margin is larger than 1, there is no gradient at all since we should normalize w . Though in SVM, we don't need to care about those data with a margin larger than 1. But if we simply ignore these data, the w may be wrongly guided: view all the data as a single batch: in one step we update according to the group of data with a margin smaller than 1, which causes more data from another group to have a margin less than one. This way, the optimization process hardly converges.

I've not seen a solution for this anywhere, and in practice, I observed such a phenomenon while using rbf kernel because the initialization makes all data have a gradient but then the gradient update falls into a loop. Possible solutions I tried are:

1) Setting a low eta:

A low learning rate makes sure margins become larger than one not at once, in order to avoid forming two groups with margins on both sides of 1.

2) Non-zero initializing:

Form a circle located at the center of data to help converge.

3) Adding a large constant in front of the kernel function. This controls how many data points have a margin larger than 1 before training.

4) Setting a better γ to make data farther from each other.

After modifications, I see the performance a little better. The loss is reducing at least.

C. Dimension Reduction

With SGD, we have already performed very fast SVM. However, I want an even faster one. The MNIST data vector is 784 dimensional, which is rather affordable. The CIFAR10 data, however, is 3072 in raw and should be over 5000 dimensional with added features. As I've shown in the last section, there may be some useless dimensions. Consider adding a random axis on linearly separable data, the added axis does not make any difference. Therefore, we want to select dimensions. The dimension reduction method we use should support reducing new data since we shouldn't perform reduction within test datasets. PCA is a suitable method, for it reserves a reduction matrix for later use, and selects the most separable features.

D. Multi Kernel

Note that the kernel function here is used somehow differently than in SMO. When we multiply x with the supporting vector, we can also view it as a higher dimensional calculation. But there is an additional limit for kernels: the range of the kernel function should be large enough for the margin to be larger than 1. What's more, there should be a learnable constant before a non-monotonic kernel. Imagine the classic case in fig 2. The constant is to determine how large the gap is since w will be normalized. b will control where the circle is.

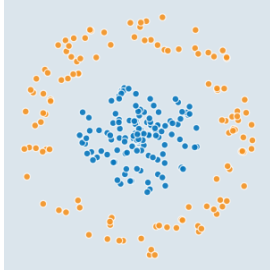


Fig. 2. Not linear separable (from tensor playground)

When we use multi-kernel, we slightly modify equation 3:

$$\mathbf{w}^T \mathbf{x} = \sum \theta_i K_i(\mathbf{w}, \mathbf{x}) \quad (7)$$

Note that for $\mathbf{w}^T \mathbf{w}$ it is the same. We only need to calculate the gradient of θ and update it.

IV. IMPLEMENTATION DETAILS

I used Python instead of C, but the time-consuming parts are implemented with Cupy, which is cuda based.

I implement my SVM like a complete project, which supports passing in all the arguments through a JSON file. The training functions are encapsulated elegantly. Because of a mess, I forget to implement the stopping condition and search for the best eta, which should be automatic in sklearn. Anyway, my default arguments are relatively nice in most cases.

A. Data Process

I used the torchvision module only to download and load data. The data is first flattened into vectors. Since they are in the range of 0 to 255 as uint8 images, I normalized them to a smaller range. Considering CIFAR10 data is complex, I added features and concatenate them together. The features I used are:

- 1) *Numerical Features*: Mean, variation, min, and max.
- 2) *Histogram*: Histogram of the whole image. I expect to get light information.
- 3) *RGB*: An RGB vector is the weight of the three colors.
- 4) *HOG*: Hog is a traditional image feature, that proved useful for classification and segmentation.

In fact, whether the features are valid is not that important, since invalid features will be deserted during PCA.

B. SVM Class

I implement SVM and multi-kernel SVM separately for they have many different parameters. When doing classification, there is a single SVC. All the classes are based on an SVM base which supports the load and saves and gradient calculation. There are two types of SVCs: when doing multi-class classification, n classes for example, you can train C_n^2 SVMs and let them vote when prediction; you can also train only n SVMs to classify one class from the rest. In my code, it is an option to do both.

C. Acceleration

I mainly use the Cupy module to perform matrix calculations on the Cuda device. However, to make my code more elegant, every concerned function supports Numpy as well. The effect of PCA is not obvious on MNIST but makes it significantly faster on CIFAR10. The PCA module is part of the homework when I was a freshman. I modify it for use.

D. Animation

I have my own tqdm-like animation module.

V. EXPERIMENTS

I don't think anyone will be happy if I put all the experiments changing all the parameters pair by pair. I will only choose those important I think. The config file in code shows only one good case, and most experiments are done in *draft.py* for there are end up thousands of experiments. The random seed is always fixed. To better record the training time, I added a warm up before training.

In table I, we see sklearn perform rather well and fast. Since Internet versions usually take ten minutes or more, it seems hard to beat libsvm.

In table II, I changed epochs to see the convergence since I didn't set a converge condition. The accuracy rises fast in the first 10 epochs, which takes only less than half a second. 100 epochs seem enough, which takes around 2.5 seconds, but gets close to the result of sklearn. The speed on CIFAR10 is fast because I set a maximum input feature dimension. If the input is too large, I use PCA to reduce the dimension. However, as we are going to see, even if we don't apply PCA, it's still fast.

In table III, I test the effects of the added features and PCA. Theoretically, applying PCA should not change performance, and the features should work alone. However, I don't design a good normalize method on the added features, and meanwhile, PCA can automatically perform some sort of normalize, so the performance gets best when applying both features and PCA.

In table IV, I changed the PCA dimension. 8 dimensions work well in 0-1 classification, but it seems more dimensions are needed in multi-classification.

In table V, I test the kernels. The linear and the default poly kernel work well, and the rbf kernel is not very stable. As I explained in the former section, I need to redesign rbf function. But I only make some failed modifications.

In table VI, I showed different modes and multi-kernel.

In the end, I polish my config and provide one of the best results of my work: I achieve 87.4% on MNIST and 51.9% on CIFAR10, which is stable under similar parameters. Because of some modification, some results of experiments may slightly change, but should not be significant.

ADDITIONAL

This report was polished by Grammarly but ChatGPT. None of the reports and code are copied Inform me if it could not pass the AI detection like GPT-Zero. Also, though my method may not be very beautiful, I base my method and code on nothing, and even ChatGPT cannot give a valid explanation or

implementation. There are lots of SMO methods on the Internet and I heard many can finish implementation within a single day, and some fused 5 or more versions of implementation. In my report, I skip what I consider naive or unnecessary. I think it's silly to copy PPT and give pages of derivation referred to somewhere. However, if you are grading according to length, inform me to submit a 10-page version with detailed SVM derivation and code explanation.

TABLE I
SKLEARN WITH RAW VECTOR

Dataset	Kernel	Performance	Time(s)
MNIST	linear	89.3%	1.71
MNIST	rbf	93.8%	6.39
CIFAR10	linear	30.7%	42.23
CIFAR10	rbf	42.7%	46.16

TABLE II
USING DEFAULT SETTINGS WITH DIFFERENT EPOCHS

Dataset	Epochs	Performance	Time(s)
MNIST	10	72.8%	0.34
MNIST	100	86.3%	2.69
MNIST	200	87.2%	5.30
CIFAR10	10	29.2%	0.31
CIFAR10	100	51.6%	2.56
CIFAR10	200	51.2%	5.35

TABLE III
USING DEFAULT SETTINGS ON CIFAR10

Features	PCA	Performance	Time(s)
-	-	33.8%	5.14
True	-	34.4%	5.48
-	True	33.8%	4.93
True	True	51.2%	4.92

TABLE IV
USING DEFAULT SETTINGS WITH DIFFERENT DIM

Dataset	PCA dim	Performance	Time(s)
MNIST	8	69.7%	5.14
MNIST	128	86.7%	5.23
MNIST	512	87.2%	4.81
MNIST	-	86.9%	5.14
CIFAR10	8	33.5%	5.21
CIFAR10	128	50.0%	5.17
CIFAR10	512	51.2%	5.19
CIFAR10	-	34.4%	5.53

REFERENCES

- [1] Cortes C, Vapnik V. Support-vector networks[J]. Machine learning, 1995, 20: 273-297.
- [2] Wang Z, Crammer K, Vucetic S. Breaking the curse of kernelization: Budgeted stochastic gradient descent for large-scale svm training[J]. The Journal of Machine Learning Research, 2012, 13(1): 3103-3131.
- [3] Lin Y, Lv F, Zhu S, et al. Large-scale image classification: fast feature extraction and svm training[C]//CVPR 2011. IEEE, 2011: 1689-1696.

TABLE V
USING DEFAULT SETTINGS WITH DIFFERENT KERNEL

Dataset	Kernel	Performance	Time(s)
MNIST	linear	83.2%	3.60
MNIST	poly	87.2%	4.81
MNIST	rbf	36.6%	6.23
CIFAR10	linear	47.5%	3.79
CIFAR10	poly	51.2%	5.19
CIFAR10	rbf	35.4%	61.74

TABLE VI
USING DEFAULT SETTINGS WITH DIFFERENT MODES

Dataset	Mode	MultiKernel	Performance	Time(s)
MNIST	one to one	-	80.5%	17.21
MNSIT	one to one	True	80.3%	26.60
MNIST	one to rest	-	87.2%	4.49
MNIST	one to rest	True	86.7%	7.07

- [4] Wijnhoven R G J, de With P H N. Fast training of object detection using stochastic gradient descent[C]//2010 20th International conference on pattern recognition. IEEE, 2010: 424-427.
- [5] Shalev-Shwartz S, Singer Y, Srebro N. Pegasos: Primal estimated sub-gradient solver for svm[C]//Proceedings of the 24th international conference on Machine learning. 2007: 807-814.
- [6] Domingos P. Every model learned by gradient descent is approximately a kernel machine[J]. arXiv preprint arXiv:2012.00152, 2020.